

Practical Assignment

Deadline: 14 November 2025

Solving the differential kinematics of a serial chain mechanism with the Brockett's exponential formula and the geometric Jacobian requires the computation of unit twists for each joint. While simple mechanisms allow for these unit twists to be computed by inspection – that is, by examining and deducing the motion of the mechanism visually – the complexity of this task increases significantly as the number of joints and degrees of freedom grows.

In such cases, an algorithmic approach is more suitable. This approach, although it may involve more complex calculations, can be standardised and automated, making it particularly useful for writing computer programmes that solve the kinematics of serial mechanisms with varying configurations.

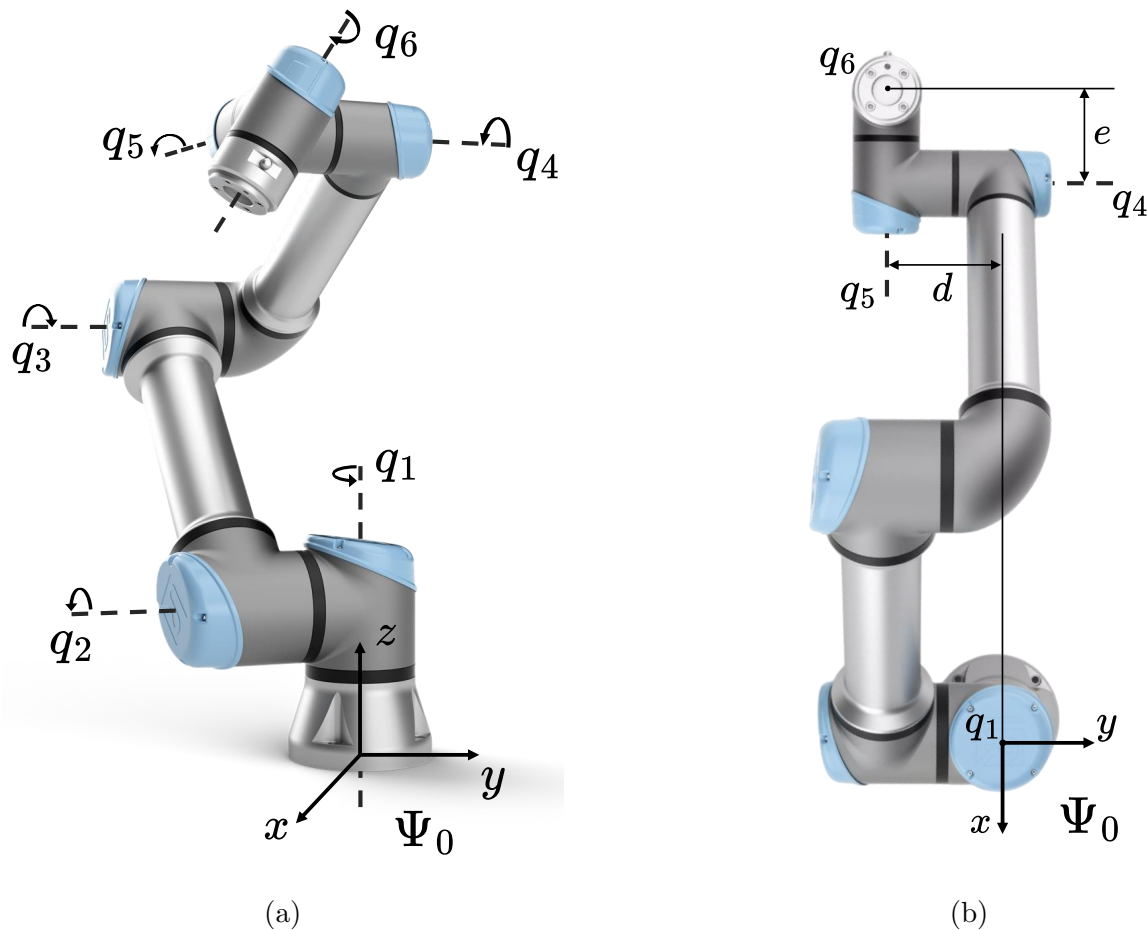


Figure 1: 6-DoF robot manipulator

Consider a 6-DoF manipulator shown in Figure 1. The robot has the following parameters:

- The distance from axis q^2 to the xy -plane of Ψ_0 is a .
- The distance between axes q^3 and q^2 is b .
- The distance between axes q^4 and q^3 is c .
- The distance between axes q^5 and q^1 is d .
- The distance between axes q^6 and q^4 is e .
- The end-effector frame Ψ_6 is fixed to the final link, its z -axis is aligned with axis q^6 (their positive directions are aligned as well), axis q^5 lies in its negative- z half-space the distance f from its xy -plane.

Part 0 — Inspection attempt

Attempt to calculate the unit twists $\hat{T}_i^{0,(i-1)}(q)$ entering the geometric Jacobian of the 6-DoF robot manipulator shown in Figure 1 by inspection, manually, and convince yourself that this task is unreasonably complicated. Do *not* include results of your attempts in the report.

Part I — Algorithm

It is possible to algorithmically compute the geometric Jacobian of a serial chain mechanism with simple joints taking as input only the constant joint unit twists $\hat{T}_i^{(i-1),(i-1)}$ expressed in the link-fixed frames and the relative configurations $H_i^{i-1}(0)$ of adjacent links in the reference configuration.

Detail, in a step-by-step manner, the algorithm you would develop to compute the geometric Jacobian of an arbitrary serial chain mechanism with simple joints. Assume you are creating a computer program capable of handling mechanisms with an arbitrary number of degrees of freedom n . Your description should include both textual and mathematical formulation of each step. Clearly indicate all the formulas involved in each step. No calculations are required here, however, it should be clear from your description, how to obtain the results knowing the necessary inputs.

1. Continue the algorithm from the following steps arriving at the expression for the geometric Jacobian $J(q) = \dots$ in the final step.
 - (i) We count links from 0 to n and joints from 1 to n . Assign a frame Ψ_i to the i -th link placing it at the $i + 1$ -th joint with the *next*, $i + 1$ -th link, such that one of its axes coincides with the axis of the joint (for $i \in \{0, \dots, n - 1\}$). For rotational and screw joints this is the axis of the rotation, and for translational joints this is any line in the direction of translation.

Note that we are free in the choice of a frame Ψ_n for the last, n -th link. It may be used to represent the pose of the end-effector.

- (ii) Specify the constant unit twists $\hat{T}_i^{(i-1),(i-1)}$ for each joint. This is a unit twist of the i -th link relative to the $(i-1)$ -th link expressed in Ψ_{i-1} . Since this frame is chosen to be aligned with this joint, for each joint type the unit twist will have the following simple form:

Suppose, without loss of generality, that the axis of a rotational joint is aligned with the x -axis of a link frame, the axis of a screw joint (with pitch λ) is aligned with the y -axis, and the axis of a translational joint is aligned with the z -axis. Then the corresponding unit twists would be:

$$\hat{T}_{\text{rot}} = \begin{pmatrix} \pm 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad \hat{T}_{\text{scr}} = \begin{pmatrix} 0 \\ \pm 1 \\ 0 \\ 0 \\ \pm \lambda \\ 0 \end{pmatrix} \quad \hat{T}_{\text{tr}} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \pm 1 \end{pmatrix}$$

We simply put a ± 1 (and $\pm \lambda$ for screws) in the corresponding slot with the sign chosen based on the positive direction of the joint coordinate q^i (use the right-hand rule for screws).

- (iii) Choose a convenient reference configuration $q = 0$.
- (iv) Specify the homogeneous matrices $H_i^{i-1}(0)$ for all adjacent frames in the chosen reference configuration based on robot's parameters (for $i \in \{1, \dots, n\}$).

Note that the last homogeneous matrix H_n^{n-1} is not used in the algorithm for differential kinematics, and thus, may not be specified, although it would still be relevant for direct kinematics.

Continue from now on...

2. Compare and contrast this algorithmic method with the method of inspection.

Part II — Implementation

In this part, you will implement in Matlab the algorithm you have described for the 6-DoF robot manipulator shown in Figure 1. Use the following robot dimensions:

$$a = 90 \quad b = 425 \quad c = 392 \quad d = 93 \quad e = 95 \quad f = 82.$$

Note that the positive directions of the joint coordinates q^i are specified in the figure by arrows. The direction of the axes is then determined by the right-hand rule.

We choose the reference configuration $q = 0$ to be:

- The axis q^2 is parallel with the y -axis of Ψ_0 as in Figure 1b;
- The axes q^3 and q^4 lie in the yz -plane of Ψ_0 , i.e. the links 2 and 3 are vertical;

- The axis q^5 is parallel with the x -axis of Ψ_0 , the direction of q^5 is aligned with the positive direction of the x -axis of Ψ_0 ;
 - The axis q^6 is parallel with the z -axis of Ψ_0 , the direction of q^6 is aligned with the positive direction of the z -axis of Ψ_0 .
 - The frame Ψ_6 is oriented the same as Ψ_0 .
1. Compute the inputs of the algorithm and provide them in the report. We advise you to assign the link frames Ψ_i in the simplest way possible, e.g. keep their orientation the same as the base frame Ψ_0 .
 2. Implement the algorithm by completing the stencil functions `direct_kinematics.m` and `get_Jacobian.m`. Split your code into elementary functions, e.g. `exp_so3.m` for the exponential map of $\mathfrak{so}(3)$. Make sure to use only the functions from the Matlab standard library, do not use toolboxes or other libraries! For example, there is no standard `skew.m` function in Matlab, so you should implement it yourself.

Add the input parameters into `robot.m` and run the script. Using the sliders, verify that the robot moves as expected, thereby testing your direct kinematics implementation.

Pay attention to the following aspects:

- The link frames must be placed as described in the algorithm, so that the input unit twists $\hat{T}_i^{(i-1),(i-1)}$ indeed have the specified form.
- All the given parameters a, b, c, d, e, f must be used.
- There is no need to shift link 2 to the side of links 1 and 3, kinematically, it is irrelevant.
- Use the reference configuration $q = 0$ as described.
- Pay attention to the signs of the unit twists, the positive directions of the joint axes are specified in the description and the figure.
- Be clever when computing the twist exponentials and inverses of homogeneous matrices.

Part III — Control

To test the implementation of the differential kinematics, we will solve a simple control problem. Unfortunately, visualising torque control in a simulation requires not only kinematical but also dynamical parameters of the robot. We will, therefore, implement velocity control instead, which requires taking the (pseudo-)inverse of the Jacobian.

The task is to follow a given trajectory $p_{\text{set}}(t)$ with the origin of the end-effector frame Ψ_6 , the orientation of the frame is irrelevant.

The current position $p_{\text{ee}}(t)$ of the end-effector origin is given by the last column of the

homogeneous matrix $H_6^0(q)$, i.e.

$$H_6^0 = \begin{pmatrix} R & p_{ee} \\ 0 & 1 \end{pmatrix}.$$

We define the error as the difference between the desired and actual positions

$$e(t) := p_{\text{set}}(t) - p_{ee}(t). \quad (1)$$

One can ensure exponential convergence of the error to zero by setting its dynamics to

$$\dot{e}(t) = -k e(t),$$

for some positive constant $k > 0$. Differentiating (1) and substituting the desired dynamics of the error, we obtain that the velocity of the end-effector should be

$$\dot{p}_{ee}(t) = \dot{p}_{\text{set}}(t) + k e(t).$$

If the geometric Jacobian $J(q)$ is invertible, the inverse differential kinematics problem can be solved by

$$T_n^{0,0} = J(q) \dot{q} \quad \Rightarrow \quad \dot{q} = J^{-1}(q) T_n^{0,0}.$$

However, the twist $T_n^{0,0}$ contains not only the linear velocity of the end-effector but also the angular velocity, which we do not wish to control. Furthermore, $v_n^{0,0}$ is not $\dot{p}_{ee}$, which is the linear velocity of the end-effector origin relative to the base frame Ψ_0 expressed in Ψ_0 . If we transform the twist $T_n^{0,0}$ to the end-effector frame Ψ_6 , we obtain the twist $T_n^{6,0}$, which contains the proper linear velocity of the end-effector relative to Ψ_0 , however, it is now expressed Ψ_6 , not in the base frame Ψ_0 . To resolve this, we can introduce another frame Ψ_e , whose origin is at the end-effector frame origin, but whose orientation is the same as Ψ_0 . Then the velocity $v_6^{e,0}$ from the twist $T_n^{e,0}$ is exactly $\dot{p}_{ee}$.

The relative configuration of Ψ_e to Ψ_0 is given by the homogeneous matrix

$$H_e^0 = \begin{pmatrix} I & p_{ee} \\ 0 & 1 \end{pmatrix}.$$

The twist $T_n^{e,0}$ can be obtained by transforming the twist $T_n^{0,0}$ to the frame Ψ_e :

$$\begin{pmatrix} \omega_n^{e,0} \\ v_n^{e,0} \end{pmatrix} = T_n^{e,0} = \text{Ad}_{H_0^e} T_n^{0,0} = \text{Ad}_{H_0^e} J(q) \dot{q}.$$

The desired linear velocity of the end-effector is then given by

$$\dot{p}_{ee} = v_n^{e,0} = \bar{J}(q) \dot{q},$$

where $\bar{J}(q)$ is the last three rows of the matrix $\text{Ad}_{H_0^e} J(q)$.

Note that $\bar{J}$ is no longer a square matrix, and thus, it does not have an inverse. We can, however, compute the [Moore–Penrose pseudo-inverse](#)

$$\bar{J}^\dagger := \bar{J}^\top (\bar{J} \bar{J}^\top)^{-1}.$$

Now, the control law for the joint velocities is

$$\dot{q} = \bar{J}^\dagger \dot{p}_{ee} = \bar{J}^\dagger (\dot{p}_{\text{set}} + k e).$$

1. Implement the function

`dq = calculate_dq(q, setpoint, dsetpoint, H0s, unit_twists)`

by modifying the stencil `calculate_dq.m`. We recommend setting $k = 50$.

Fill the robot parameters in the function `robot_control.m` and run the script. Verify that the robot tries to follow the trajectory given by

$$\begin{cases} x = 600 \\ y = 200 \cos(t) \\ z = 400 + 200 \sin(2t) \end{cases},$$

that draws a butterfly-like curve in the yz -plane.

Submission

Submit a .pdf report containing the answers to the questions and the Matlab code of the functions `direct_kinematics.m`, `get_Jacobian.m`, and of any complementary functions you have implemented. Put all the code files in a single .zip archive together with the report file. The functions `robot.m` and `robot_control.m` should be executable and reproduce the results described in the report. **Make sure not to create any subfolders, all the files should be in the root of the .zip archive.**